

Project Report: AI Resume Analyzer & Job Matching Dashboard

This report provides a comprehensive summary of the architecture, database design, key features, technical decisions, and validation results of the **AI Resume Analyzer & Job Matching System**.

1. Project Overview

The AI Resume Analyzer is a fully offline, local application designed to help job seekers optimize their resumes against modern Applicant Tracking Systems (ATS) and find matching career opportunities. The application parses a resume PDF, extracts technical/soft skills, evaluates structural formatting compliance, and calculates a multidimensional ATS score. It also suggests tailored, action-oriented bullet points for missing skills and recommends careers using semantic search.

2. System Architecture

The project is structured into a clean, decoupled architecture consisting of a Python Flask backend and a responsive vanilla HTML/CSS/JavaScript frontend:

A. Frontend Component

- **Technology:** HTML5, Vanilla CSS3 (Custom design system), JavaScript (ES6+), and **Chart.js** for visualization.
- **Layout:** A premium, light-mode SaaS dashboard with a collapsible history sidebar, interactive cards, animated circular SVG gauges, and tabbed panels.
- **File Structure:**
- **index.html:** HTML skeleton including structural elements, modal triggers, and result sections.
- **style.css:** Custom CSS variables, gradients, card overlays, transitions, animations, and responsive grids.
- **script.js:** Frontend controller handling drag-and-drop actions, API fetch calls, history caching, chart loading, and modal state management.

B. Backend Component

- **Technology:** Python 3, Flask, SQLite3, spaCy, and Sentence Transformers (**all-MiniLM-L6-v2**).
- **File Structure:**

- **app.py**: Flask API server, handling routing, CORS policies, and uploads.
 - **resume_parser.py**: Extracting structure, text, page boundaries, word count, contact details, and essential sections.
 - **skills_extractor.py**: Regex-based dictionary phrase matcher for extracting overlapping and missing skills.
 - **matcher.py**: Scoring module using cosine similarity for semantic matching and job recommendations.
 - **ai_feedback.py**: Categorized suggestion builder for layout, skills gap, and content impact.
 - **database.py**: SQLite data layer for managing database persistence.
-

3. Database Schema

The local SQLite database (**database.db**) stores previous scans and job listings to provide full offline capability.

A. scans Table

Stores the results of every analyzed resume for historical tracking and quick reload:

Column	Type	Description
id	INTEGER	Primary Key (Autoincrement)
filename	TEXT	The name of the uploaded PDF resume
job_description	TEXT	The pasted target job description text
ats_score	REAL	The overall weighted ATS compatibility score (0-100)
similarity_score	REAL	Semantic cosine similarity score (50% weight)
skill_score	REAL	Skill match density score (30% weight)
formatting_score	REAL	Structural checklist compliance score (20% weight)
missing_skills	TEXT (JSON)	JSON array of skills required by JD but missing in resume
matched_skills	TEXT (JSON)	JSON array of overlapping skills present in both
feedback	TEXT (JSON)	JSON object containing categorized AI feedback arrays
recommended_jobs	TEXT (JSON)	JSON array of recommended jobs returned by matcher
created_at	TIMESTAMP	Date and time of scan (Defaults to CURRENT_TIMESTAMP)

B. jobs Table

Stores seeded mock job listings for the semantic recommendation engine:

Column	Type	Description
id	INTEGER	Primary Key (Autoincrement)
title	TEXT	Job Title (e.g. Python Backend Developer, React Engineer)
company	TEXT	Company name
location	TEXT	Location status (On-site, Hybrid, Remote)
description	TEXT	Full description text used for semantic matching
skills	TEXT (JSON)	JSON array of key skills expected for this job
salary	TEXT	Salary bracket

4. Key Implemented Features

■ Multi-Faceted ATS Scoring

Rather than a simple keyword count, the system calculates a weighted ATS score based on:

1. **Semantic Fit (50%)**: Captures contextual alignment between the resume and the job description using Sentence Transformers.
2. **Skill Match (30%)**: Measures the density of matched technical and soft skills.
3. **Formatting Quality (20%)**: Checks for page length, word count limits, contact info, and structural sections (Experience, Projects, Education).

■ Interactive Skills Analysis Tabs

Users can toggle between **Matched Skills** (green) and **Missing Skills** (orange/red) to quickly review keyword gaps.

■ Resume Rewriter Helper Modal

- Missing skill tags are interactive (clickable with cursor pointer indicators and hover highlights).
- Clicking a missing tag displays a pop-up modal showing a **tailored, action-oriented bullet point** describing an achievement with that skill (e.g. implementing Python backend microservices, or building responsive UI layouts in React).
- Features one-click **Copy to Clipboard** with a green success badge confirmation that fades out after 2 seconds.

■ Categorized AI Feedback Accordions

Presents constructive warnings and tips in three distinct categories:

- **Skills Optimization:** Specific actions for filling technical skill gaps.
- **Layout & Format Checks:** Identifies missing sections or page limits.
- **Content Impact & Quality:** Flagging usage of passive voice, weak action verbs, or missing numerical metrics.

■ Semantic Career Opportunities

Recommends jobs from the local database by comparing the candidate's resume text against descriptions using deep semantic similarity. Clicking any job card automatically pre-fills the Job Description input for a rapid re-test.

■ Sidebar Scan History & Toggle

- Persistent left sidebar lists all past scans with color-coded score badges. Clicking a history item instantly loads and visualizes its full dashboard state from the database.
- Features a trash icon to delete specific scan records.
- Collapsible sidebar button lets the user hide the history drawer to maximize dashboard space, automatically resizing the Chart.js canvas.

■ Premium SaaS Visual Aesthetics

- Modern layout with custom typography (Outfit and Plus Jakarta Sans).
 - Sleek Indigo-to-Sky gradient header title and sidebar branding.
 - Glassmorphic cards utilizing border shadows and hover scaling.
 - Glowing accent border lines under the top navigation and styled status indicators.
-

5. Technical Decisions & Rationale

- **Phrase-Based Skill Boundary Extraction:** Simple tokenization misses multi-word phrases (e.g. "machine learning", "project management") and symbols (e.g. "C++", "C#", "Node.js"). The custom **skills_extractor.py** compiles regex patterns with word boundaries and manual symbol patterns to ensure accurate extraction.
- **Sentence Transformers (all-MiniLM-L6-v2):** Using an offline local embedding model allows the application to execute semantic similarity searches rapidly, privately, and for free, without needing OpenAI or other external API keys.
- **Client-Side Rewriter Bullet Mapping:** High-quality, metric-based client-side templates are mapped to common keywords in the frontend. This provides instant results for the user and maintains full offline status without calling LLM endpoints.
- **SQLite Dynamic Schema Migration:** To support schema modifications (like adding **recommended_jobs** to the history log) without breaking existing user instances, **database.py** automatically drops and re-initializes tables when columns mismatch on startup.

6. Verification and Testing Results

- **Syntactical Validations:** The JavaScript code was checked and compiled via **node -c**, confirming zero syntax errors.
- **End-to-End Test Execution:**
 - Tested the pipeline using a mock PDF resume containing standard sections (Contact, Experience, Projects, Skills, Education).
 - Run against a Python Backend Developer description, yielding:
 - **ATS Score: 72.82%**
 - **Semantic Cosine Similarity: 60.65%**
 - **Missing Skills:** ['django', 'postgresql']
 - **Matched Skills:** ['docker', 'flask', 'git', 'machine learning', 'python', 'sql']
 - Clicking on missing skill tags opens the modal displaying the correct customized bullet point text, copying it updates the clipboard, and all close triggers behave correctly.
 - DB persistence and reload functionalities behave as expected with no schema locks.